

Architecture & Trade-offs

Account Ledger Core | Architecture review

Append-only at scale

The implementation retains every source event and every derived posting in in-memory lists. Balance lookup scans those entries, so a balance query is $O(n)$ in the number of postings. Fee assessment repeats that scan across dates, and the replay retains authorization state, error history, fee metadata, and interest details without eviction. At 100x volume, CPU spent repeatedly scanning entries and the unbounded in-process event/derived-entry collections are the first practical limits; process restart also loses the entire ledger.

The cheapest structural change is to index entries by account and value date, while retaining an append-only journal as the source of truth. That avoids scanning unrelated accounts and gives an $O(\log n)$ or near- $O(1)$ route to a date's opening/closing balance, depending on the chosen index. It defers the memory ceiling but does not solve durability or multi-writer correctness. The next step would be partitioned durable event storage plus reproducible checkpoints/snapshots; snapshots are rebuildable projections, never replacements for the journal.

Value-dated entries in production

Value dating separates when the institution accepts/records an instruction from the date on which it affects account value. Backdating can alter historical balance views, interest, fee eligibility, statements, customer notices, reconciliation, liquidity/settlement positions, and downstream regulatory or management reporting. The operational surface includes who may enter a backdated posting, which date ranges and reason codes are allowed, how prior statements and calculations are corrected, and how a reviewer can reconstruct what was known at each point in time.

Before a UAE-licensed bank went live with this capability, I would require a maker-checker control for backdated postings: capture the reason and supporting reference, preview affected balances/fees/interest and downstream reports, and require an authorized second person to approve outside-policy dates. The immutable event record should preserve both booking and value timestamps, actor, approver, and correction linkage. Exact regulatory retention and reporting obligations would be validated with the bank's compliance and finance owners rather than inferred from this toy model.

Authorization lifecycle

An approved authorization in this model has no non-settlement termination path: it remains active until a matching settlement. The only non-settlement terminal outcome implemented is a declined authorization request, where no hold is created. In the supplied stream Auth-B is declined for insufficient available balance. An unknown or invalid settlement is a rejected operation, not an end-state transition for an existing authorization.

The implementation releases Auth-A's unused 15 only because its matching settlement is accepted. Production products also need explicit non-settlement release events for real lifecycle cases:

1. Expiry: the merchant does not capture before the network/product deadline. Mandate an idempotent expiry event that releases only the unconsumed hold, records the policy deadline, and reconciles late captures rather than silently dropping the hold.
2. Customer/merchant cancellation or void: the purchase is abandoned before capture. Mandate a referenced cancellation event, authorization of the caller, and release of the remaining hold; do not book a ledger debit.
3. Issuer/network reversal: the rail invalidates an authorization or sends a reversal/void message. Mandate matching on the rail reference, duplicate-safe processing, an append-only release record, and exception handling when the released amount differs from the remaining hold.
4. Controlled operational release: an investigation confirms an erroneous or stranded hold. Mandate reason codes, maker-checker approval, evidence retention, a release event, and reconciliation with the payment rail.

Partial capture and release of the unused remainder is a matching-settlement path, not a separate non-settlement ending. The toy model also omits multiple captures and settlement reversals; production state transitions and deadlines must be specified per payment rail/product.

What was cut and why

- Durability and recovery: memory-only state keeps the exercise small. Production risk: process loss destroys balances and audit history. Add a durable append-only journal, backups, recovery tests, and rebuildable snapshots.
- Concurrency and idempotency: the supplied single ordered stream avoids races. Production risk: duplicate delivery or concurrent authorization can overspend. Add idempotency keys, atomic balance/hold reservation, sequencing, and concurrency controls.
- External API, database, and UI: intentionally absent per scope. Production risk: no integration, operational tooling, maker-checker workflow, or customer servicing. Add only behind controlled service boundaries.
- Single account/currency per movement and no FX: the scenario specifies fixed account currencies. Production risk: currency mismatch or implicit conversion could corrupt amounts. Validate currency on every event; model FX as linked, explicit legs with approved rates.
- Restricted authorization lifecycle: one authorization can have a single settlement; there is no timeout, cancellation, multiple capture, or settlement reversal. Production risk: real payment flows will strand or incorrectly release holds. Define rail-specific state machines and timeout reconciliation.
- One simple fee rule and no tax/product tiers: only the stated AED daily fee is implemented. Production risk: exemptions, waivers, customer disclosures, and product-specific rules are absent. Use versioned policy and effective dates with audit records.
- Simple daily interest and six-day horizon: there are no calendar conventions, rate changes, compounding schedules, or statement cutoffs. Production risk: contractual interest may be wrong. Use approved day-count conventions, effective-dated rates, independent reconciliation, and controlled rounding.
- Illustrative backdating policy: late entries can lead to append-only fees; no compensating fee reversal process is supplied. Production risk: an incorrect or subsequently corrected fee remains charged. Require controlled fee correction events, customer remediation, approval, and auditable recalculation policy.
- No regulatory/accounting integration: the model has no GL, settlement, AML, sanctions, or reporting interfaces. Production risk: the ledger is not a complete regulated-bank control environment. Reconcile to the general ledger and payment rails, apply access controls, retain evidence, and obtain compliance sign-off.